WEEK – 01

a) FCFS:

```
#include <stdio.h>
struct process {
    int pid, at, bt, ct, tat, wt, rt;
};
int main() {
    int n, i, j;
    float avgTAT = 0, avgWT = 0, avgRT = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct process p[n], temp;
    for(i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter AT and BT for P%d: ", i+1);
        scanf("%d %d", &p[i].at, &p[i].bt);
    }
    for(i = 0; i < n-1; i++) {
        for(j = i+1; j < n; j++) {
            if(p[i].at > p[j].at) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    int time = 0;
    for(i = 0; i < n; i++) {

        if(time < p[i].at)
            time = p[i].at;

        p[i].rt = time - p[i].at;
        p[i].wt = time - p[i].at;
        time += p[i].bt;
        p[i].ct = time;
        p[i].tat = p[i].ct - p[i].at;
```

```
        avgTAT += p[i].tat;
        avgWT += p[i].wt;
        avgRT += p[i].rt;
    }
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct,
        p[i].tat, p[i].wt, p[i].rt);
    }
    printf("\nAverage TAT = %.2f", avgTAT/n);
    printf("\nAverage WT = %.2f", avgWT/n);
    printf("\nAverage RT = %.2f\n", avgRT/n);

    printf("\nGantt Chart:\n|");
    for(i = 0; i < n; i++) {
        printf(" P%d |", p[i].pid);
    }
    printf("\n%d", p[0].at);
    for(i = 0; i < n; i++) {
        printf("    %d", p[i].ct);
    }
    return 0;
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS C:\Users\SHALINI K A\Desktop\OS LAB> gcc FCFS.c -o FCFS
PS C:\Users\SHALINI K A\Desktop\OS LAB> ./FCFS
Enter number of processes: 5
Enter AT and BT for P1: 2
2
Enter AT and BT for P2: 0
1
Enter AT and BT for P3: 2
3
Enter AT and BT for P4: 3
5
Enter AT and BT for P5: 4
4

PID     AT      BT      CT      TAT     WT      RT
P2      0       1       1       1       0       0
P1      2       2       4       2       0       0
P3      2       3       7       5       2       2
P4      3       5       12      9       4       4
P5      4       4       16      12      8       8

Average TAT = 5.80
Average WT = 2.80
Average RT = 2.80

Gantt Chart:
| P2 | P1 | P3 | P4 | P5 |
0    1    4    7    12    16
```

WEEK-01

1. Write a C program to simulate the following CPU scheduling algorithm to find turnaround & waiting time

a) FCFS

```
#include <stdio.h>
int main(){
    int i,j,n,temp;
    int P[20], at[20], bt[20], tat[20], wt[20],
        ct[20], rt[20];
    float avg_tat=0, avg_wt=0, avg_rt=0;

    printf("Enter the number of processes:");
    scanf("%d", &n);

    for (i=0; i<n; i++){
      P[i]=i+1;
      printf("Enter the arriving time=");
      scanf("%d", &at[i]);
    }
    for (i=0; i<n; i++){
      for(j=i+1; j<n; j++){
        if (at[i] > at[j]){
          temp = at[i]; at[i]=at[j]; at[j]
            =temp;
          temp= bt[i]; bt[i]=bt[j]; bt[j]=
            temp;
          temp=P[i]; P[i]=P[j]; P[j]=temp;
```

REDMI NOTE 9
48MP QUAD CAMERA

```
    }}}
    ct[0]=at[0]+bt[0];
    for(i=1; i<n; i++){
      if(ct[i-1] < at[i])
        ct[i]=at[i]+bt[i]
      else
        ct[i] = ct[i-1]+ bt[i];
    }
    printf("P\t AT \t BT \t CT \t TAT \t WT \t RT \n");

    for (i=0; i<n; i++){
      tat[i] = ct[i] - at[i];
      wt[i] = tat[i] - bt[i];
      rt[i] = ct[i] - bt[i] - at[i];

      avg_tat += tat[i];
      avg_wt += wt[i];
      avg_rt += rt[i];

      printf("%d \t %d \t %d \t %d \t %d \t %d \t %d \n",
        P[i], at[i], bt[i], ct[i], tat[i], wt[i],
        rt[i]);
    }
    printf("Average waiting time=%.2f\n", avg_wt/n)
    printf("Average turnaround time=%.2f\n", avg_tat/n
    printf("Average response time=%.2f\n", avg_rt/n
    return 0;
}
```

output

```
Enter the number of processes = 5
Enter the arriving time = 2
Enter the burst time = 2
Enter the arriving time = 0
Enter the burst time = 1
Enter the arriving time = 2
Enter the burst time = 3
Enter the arriving time = 3
Enter the burst time = 5
Enter the arriving time = 4
Enter the burst time = 4
```

P	AT	BT	CT	TAT	WT	RT
1	2	2	4	2	0	0
2	0	1	1	1	0	0
3	2	3	7	5	2	2
4	3	5	12	9	4	4
5	4	4	16	12	8	8

Average waiting time = 2.8
Average Turn around time = 5.8
Average response time = 2.8

Ready queue

P2 P1 P3 P4 P5

Gantt chart

P2		P1	P3	P4	P5
0	1	2	4	7	12	16

b) SJF Preemptive Non Preemptive Scheduling

```
#include <stdio.h>
int main()
{
    int n,i,choice;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    int at[20],bt[20],ct[20],tat[20],wt[20],rt[20];
    int rem[20];

    for(i=0;i<n;i++)
    {
        printf("Enter AT and BT for P%d: ",i+1);
        scanf("%d%d",&at[i],&bt[i]);
        rem[i]=bt[i];
        rt[i]=-1;
    }
    printf("\n1. SJF Non Preemptive");
    printf("\n2. SJF Preemptive");
    printf("\nEnter choice: ");
    scanf("%d",&choice);

    float avgTAT=0,avgWT=0,avgRT=0;
    if(choice==1)
    {
        int completed=0,time=0,done[20]={0};

        printf("\nGantt Chart:\n|");

        while(completed<n)
        {
            int idx=-1;
            int min=9999;

            for(i=0;i<n;i++)
            {
```

```
            if(at[i]<=time && done[i]==0 && bt[i]<min)
            {
                min=bt[i];
                idx=i;
            }
        }
        if(idx==-1)
        {
            time++;
            continue;
        }

        printf(" P%d |",idx+1);

        rt[idx]=time-at[idx];
        wt[idx]=time-at[idx];
        time+=bt[idx];
        ct[idx]=time;
        tat[idx]=ct[idx]-at[idx];

        done[idx]=1;
        completed++;

        avgTAT+=tat[idx];
        avgWT+=wt[idx];
        avgRT+=rt[idx];
    }
}

else
{
    int completed=0,time=0;

    printf("\nGantt Chart:\n|");

    while(completed<n)
    {
        int idx=-1;
        int min=9999;
```

```
        for(i=0;i<n;i++)
        {
            if(at[i]<=time && rem[i]>0 && rem[i]<min)
            {
                min=rem[i];
                idx=i;
            }
        }
        if(idx==-1)
        {
            time++;
            continue;
        }

        printf(" P%d |",idx+1);

        if(rt[idx]==-1)
            rt[idx]=time-at[idx];

        rem[idx]--;
        time++;

        if(rem[idx]==0)
        {
            completed++;

            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];

            avgTAT+=tat[idx];
            avgWT+=wt[idx];
            avgRT+=rt[idx];
        }
    }
}

printf("\n\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(i=0;i<n;i++)
```

```
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
    }

    printf("\nAverage TAT = %.2f",avgTAT/n);
    printf("\nAverage WT  = %.2f",avgWT/n);
    printf("\nAverage RT  = %.2f\n",avgRT/n);

    return 0;
}
```

Output:

SJF Non Preemptive

```
PS C:\Users\SHALINI K A\Desktop\OS LAB> gcc SJF.c -o SJF
PS C:\Users\SHALINI K A\Desktop\OS LAB> ./SJF
Enter number of processes: 5
Enter AT and BT for P1: 2
1
Enter AT and BT for P2: 1
5
Enter AT and BT for P3: 4
1
Enter AT and BT for P4: 0
6
Enter AT and BT for P5: 2
3

1. SJF Non Preemptive
2. SJF Preemptive
Enter choice: 1

Gantt Chart:
| P4 | P1 | P3 | P5 | P2 |

PID     AT      BT      CT      TAT     WT      RT
P1      2       1       7       5       4       4
P2      1       5       16      15      10      10
P3      4       1       8       4       3       3
P4      0       6       6       6       0       0
P5      2       3       11      9       6       6

Average TAT = 7.80
Average WT  = 4.60
Average RT  = 4.60
```

SJF Preemptive

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL

PS C:\Users\SHALINI K A\Desktop\OS LAB> gcc SJF.c -o SJF
PS C:\Users\SHALINI K A\Desktop\OS LAB> ./SJF
Enter number of processes: 5
Enter AT and BT for P1: 2
1
Enter AT and BT for P2: 1
5
Enter AT and BT for P3: 4
1
Enter AT and BT for P4: 0
6
Enter AT and BT for P5: 2
3

1. SJF Non Preemptive
2. SJF Preemptive
Enter choice: 2

Gantt Chart:
| P4 | P2 | P1 | P5 | P3 | P5 | P5 | P2 | P2 | P2 | P2 | P4 | P4 | P4 | P4 | P4 |

PID     AT      BT      CT      TAT     WT      RT
P1      2       1       3       1       0       0
P2      1       5       11      10      5       0
P3      4       1       5       1       0       0
P4      0       6       16      16      10      0
P5      2       3       7       5       2       1

Average TAT = 6.60
Average WT  = 3.40
Average RT  = 0.20
```

Page No.
Date:

Ready queue

P4 P2 P1 P5 P3

Gantt Chart

P4	P4	P1	P5	P3	P5	P4
0	11	2	3	4	5	7

Average Turn around time = 6.6

Average waiting time = 3.4

Average response time = 2.2

b) SJF Scheduling (Preemptive & Non-Preemptive)

```
#include <stdio.h>
int main(){
    int n, i, j, choice;
    int at[20], bt[20], ct[20], tat[20], wt[20],
     rt[20];
    int remain, time = 0, smallest;
    int rt_bt[20];
    float avg_tat=0, avg_wt=0, avg_rt=0;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Arrival time & Burst Time:\n
    for(i=0; i<n; i++){
        printf("P%d AT: ", i+1);
        scanf("%d", &at[i]);
        printf("P%d BT: ", i+1);
        scanf("%d", &bt[i]);
        rt_bt[i] = bt[i];
    }
    printf("\n1. SJF Non Preemptive\n2. SJF
        Preemptive\n Enter choice: ");
    scanf("%d", &choice);
    if(choice==1){
        int done[20]={0}, count=0, min;
        while(count<n){
            min=999;
            int index=-1;
            for(i=0; i<n; i++){
                if(at[i]<=time && done[i]==0
                    && bt[i]<min){
```

```
                    min=bt[i];
                    index=i;
                }
            }
            if(index==-1){
                time++;
            }
            else{
                rt[index]=time-at[index];
                time+=bt[index];
                ct[index]=time;
                tat[index]=ct[index]-at[index];
                wt[index]=tat[index]-bt[index];
                done[index]=1;
                count++;
            }
        }
    }
    else if(choice==2){
        remain=n;
        int start[20];
        for(i=0;i<n; i++){
            start[i]=-1;
        }
        while(remain>0){
            smallest=-1;
            for(i=0; i<n; i++){
                if(at[i]<=time && rt_bt[i]>0){
                    if(smallest==-1 || rt_bt[i]<
                        rt_bt[smallest])
                        smallest=i;
```

```
            if(smallest==-1){
                time++;
                continue;
            }
            if(start[smallest]==-1){
                start[smallest]=time;
                rt[smallest]=time-at[smallest];
            }
            rt_bt[smallest]--;
            time++;

            if(rt_bt[smallest]==0){
                ct[smallest]=time;
                tat[smallest]=ct[smallest]-at[smallest]
                wt[smallest]=tat[smallest]-bt[smallest]
                remain--;
            }
        }
    }
    printf("\nProcess\t AT\t BT\t CT\t TAT\t WT\t
        RT\n");
    for(i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1, at[i], bt[i], ct[i], tat[i],
            wt[i], rt[i]);
    }
    avg_tat += tat[i];
    avg_wt += wt[i];
    avg_rt += rt[i];
    }
    avg_tat /=n;
    avg_wt /=n;
    avg_rt /=n;
    printf("\nAverage TAT time=%.2f",
        avg_tat);
    printf("\nAverage WT %.2f",
        avg_wt);
    return 0;
}
```

Output

```
Enter number of processes: 5
Enter Arrival time & Burst time:
P1 AT: 2        P1 BT: 1
P2 AT: 1        P2 BT: 5
P3 AT: 4        P3 BT: 1
P4 AT: 0        P4 BT: 6
P5 AT: 2        P5 BT: 3

1. SJF Non Preemptive
```

Process	AT	BT	CT	TAT	WT	RT
P1	2	1	7	5	4	4
P2	1	5	16	15	10	10
P3	4	1	8	4	3	3
P4	0	6	6	6	0	0
P5	2	3	11	9	6	6

Ready queue

AT -> | P4 | P2 | P1 | P5 | P3 |

BT -> | P4 | P1 | P3 | P5 | P2 |

Gantt chart

P4	P1	P3	P5	P2

0 6 7 8 11 16

Average Turnaround Time = 7.8

Average waiting time = 4.6

Average response Time = 4.6

2. SJF Preemptive

Process	AT	BT	CT	TAT	WT	RT
P1	2	1	3	1	0	0
P2	1	5	16	15	10	10
P3	4	1	5	1	0	0
P4	0	6	11	11	5	0
P5	2	3	7	5	2	1

C) Round Robin Preemptive Scheduling

```
#include<stdio.h>
int main()
{
    int n,i,tq;
    int at[20],bt[20],ct[20],tat[20],wt[20],rt[20];
    int rem_bt[20],start[20];
    int queue[100];
    int front=0,rear=0;
    int time=0,completed=0;
    float avg_tat=0,avg_wt=0,avg_rt=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("P%d AT: ",i+1);
        scanf("%d",&at[i]);

        printf("P%d BT: ",i+1);
        scanf("%d",&bt[i]);

        rem_bt[i]=bt[i];
        start[i]=-1;
    }
    printf("Enter Time Quantum: ");
    scanf("%d",&tq);
```

```
int visited[20]={0};

printf("\nGantt Chart:\n|");

while(completed<n)
{
    for(i=0;i<n;i++)
    {
        if(at[i]<=time && visited[i]==0)
        {
            queue[rear++]=i;
            visited[i]=1;
        }
    }
    if(front==rear)
    {
        time++;
        continue;
    }
    int p=queue[front++];
    if(start[p]==-1)
    {
        start[p]=time;
        rt[p]=time-at[p];
    }
    int exec;
```

```
if(rem_bt[p] > tq)
    exec = tq;
else
    exec = rem_bt[p];

printf(" P%d |",p+1);

time += exec;
rem_bt[p] -= exec;
for(i=0;i<n;i++)
{
    if(at[i]<=time && visited[i]==0)
    {
        queue[rear++]=i;
        visited[i]=1;
    }
}
if(rem_bt[p] > 0)
{
    queue[rear++]=p;
}
else
{
    ct[p]=time;
    tat[p]=ct[p]-at[p];
    wt[p]=tat[p]-bt[p];
    completed++;
```

```
    }
}
printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);

    avg_tat += tat[i];
    avg_wt += wt[i];
    avg_rt += rt[i];
}
avg_tat /= n;
avg_wt /= n;
avg_rt /= n;
printf("\nAverage TAT = %.2f",avg_tat);
printf("\nAverage WT  = %.2f",avg_wt);
printf("\nAverage RT  = %.2f\n",avg_rt);
return 0;}
```

Output

```
PS C:\Users\SHALINI K A\Desktop\OS LAB> gcc RoundRobin.c -o RoundRobin
PS C:\Users\SHALINI K A\Desktop\OS LAB> ./RoundRobin
Enter number of processes: 5
P1 AT: 0
P1 BT: 8
P2 AT: 5
P2 BT: 2
P3 AT: 1
P3 BT: 7
P4 AT: 6
P4 BT: 3
P5 AT: 8
P5 BT: 5
Enter Time Quantum: 3

Gantt Chart:
| P1 | P3 | P1 | P2 | P4 | P3 | P5 | P1 | P3 | P5 |

Process AT      BT      CT      TAT     WT      RT
P1      0       8       22      22      14      0
P2      5       2       11      6       4       4
P3      1       7       23      22      15      2
P4      6       3       14      8       5       5
P5      8       5       25      17      12      9

Average TAT = 15.00
Average WT  = 10.00
Average RT  = 4.00
```

c) Round Robin (Preemptive)

```
#include<stdio.h>
int main(){
    int n,i,time=0, remain;
    int at[20], bt[20], rt[20], ct[20], tat[20],
        wt[20], resp[20];
    int tq, flag=0;
    float avg_tat=0, avg_wt=0, avg_rt=0;

    Printf("Enter number of Processes: ");
    Scanf("%d", &n);
    Printf("Enter Arrival Time & Burst time:\n");
    for(i=0; i<n; i++){
        Printf("P%d AT: ", i+1);
        Scanf("%d", &at[i]);
        Printf("P%d BT: ", i+1);
        Scanf("%d", &bt[i]);
        rt[i] = bt[i];
        resp[i] = -1;
    }
    Printf("Enter Time Quantum: ");
    Scanf("%d", &tq);
    remain = n;
    while(remain!=0){
        flag=0;
        for(i=0; i<n; i++){
            if(at[i] <= time && rt[i] > 0){
                if(resp[i] == -1)
                    resp[i] = time - at[i];
                if(rt[i] < tq){
                    time += rt[i];
                    rt[i] = 0;
                    ct[i] = time;
                    remain--;
                    flag=1;
                }
                else{
                    rt[i] -= tq;
                    time += tq;
                    flag=1;
                }
            }
        }
        if(flag==0)
            time++;
    }
    ~~Printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",~~
    ~~i+1, at[i], bt[i], ct[i], tat[i], wt[i], resp[i]);~~
    Printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\
        n");
    for(i=0; i<n; i++){
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    Printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], ct[i], tat[i], wt[i], resp[i]);
        avg_tat += tat[i];
        avg_wt += wt[i];
        avg_rt += resp[i];
    }
```

```
printf("\n Average TAT = %.2f", avg_tat/n);
printf("\n Average WT = %.2f, avg_wt/n);
printf("\n Average RT = %.2f, avg_rt/n);
return 0;
}
```

Output

```
Enter number of Processes : 3
Enter Arrival Time & Burst Time;
P1 AT: 0      P1 BT: 8
P2 AT: 5      P2 BT: 2
P3 AT: 1      P2 BT: 7
P4 AT: 6      P4 BT: 3
P5 AT: 8      P5 BT: 5
Enter time quantum: 3
```

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	22	22	14	0
P2	5	2	11	6	4	4
P3	1	7	23	22	15	2
P4	6	3	14	8	5	5
P5	8	5	25	17	12	9

Ready queue

P1	P3	P1	P2	P4	P3	P5	P1	P3	P5

Gantt Chart

P1	P3	P1	P2	P4	P3	P5	P1	P3	P5

0 3 6 9 11 14 17 20 22 23 25

Average TAT = 10
Average WT = 15
Average RT = 4

D) Priority Scheduling(Preemptive and Non Preemptive)

```
#include <stdio.h>
#include <limits.h>
#define MAX 100
typedef struct {
    int pid, at, bt, pr;
    int ct, tat, wt, rt;
    int remaining;
    int started;
} Process;

void calculateTimes(Process p[], int n) {
    float totalTAT = 0, totalWT = 0, totalRT = 0;
    for (int i = 0; i < n; i++) {
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;

        totalTAT += p[i].tat;
        totalWT += p[i].wt;
        totalRT += p[i].rt;
    }
    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
                p[i].pid, p[i].at, p[i].bt, p[i].pr,
                p[i].ct, p[i].tat, p[i].wt, p[i].rt);
    }
```

```
    printf("\nATAT = %.2f", totalTAT / n);
    printf("\nAWT  = %.2f", totalWT / n);
    printf("\nART  = %.2f\n", totalRT / n);
}
void priorityNonPreemptive(Process p[], int n) {
    int completed = 0, time = 0;
    int visited[MAX] = {0};

    printf("\nGantt Chart:\n|");

    while (completed < n) {
        int idx = -1;
        int highest = INT_MAX;

        for (int i = 0; i < n; i++) {
            if (p[i].at <= time && !visited[i]) {
                if (p[i].pr < highest) {
                    highest = p[i].pr;
                    idx = i;
                }
            }
        }
        if (idx != -1) {
            printf(" P%d |", p[idx].pid);
            p[idx].rt = time - p[idx].at;
            time += p[idx].bt;
            p[idx].ct = time;
```

```
            visited[idx] = 1;
            completed++;
        } else {
            printf(" Idle |");
            time++;
        }
    }
    printf("\n");
    calculateTimes(p, n);
}
void priorityPreemptive(Process p[], int n) {
    int completed = 0, time = 0;
    int last = -1;

    for (int i = 0; i < n; i++) {
        p[i].remaining = p[i].bt;
        p[i].started = 0;
    }
    printf("\nGantt Chart:\n|");

    while (completed < n) {
        int idx = -1;
        int highest = INT_MAX;
        for (int i = 0; i < n; i++) {
            if (p[i].at <= time && p[i].remaining > 0) {
                if (p[i].pr < highest) {
```

```
            highest = p[i].pr;
            idx = i;
        }
    }
}
if (idx != -1) {

    if (last != idx) {
        printf(" P%d |", p[idx].pid);
        last = idx;
    }
    if (!p[idx].started) {
        p[idx].rt = time - p[idx].at;
        p[idx].started = 1;
    }
    p[idx].remaining--;
    time++;
    if (p[idx].remaining == 0) {
        p[idx].ct = time;
        completed++;
    }
} else {
    if (last != -2) {
        printf(" Idle |");
        last = -2;
    }
    time++;
```

```
        }
    }
    printf("\n");
    calculateTimes(p, n);
}
int main() {
    int n, choice;
    Process p[MAX], temp[MAX];

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("\nEnter AT, BT, Priority for P%d: ", i + 1);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
        p[i].pid = i + 1;
    }

    printf("\n1. Non-Preemptive Priority Scheduling");
    printf("\n2. Preemptive Priority Scheduling");
    printf("\nEnter your choice: ");
    scanf("%d", &choice);
    for (int i = 0; i < n; i++) {
        temp[i] = p[i];
    }
    if (choice == 1) {
        priorityNonPreemptive(temp, n);
    } else if (choice == 2) {
```

```
        priorityPreemptive(temp, n);
    } else {
        printf("Invalid choice!\n");
    }
    return 0;
}
```

Output:

Preemptive:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL                    powershell

PS C:\Users\SHALINI K A\Desktop\OS LAB> gcc PriorityScheduling.c -o PriorityScheduling
PS C:\Users\SHALINI K A\Desktop\OS LAB> ./PriorityScheduling
Enter number of processes: 7

Enter AT, BT, Priority for P1: 0
8
3

Enter AT, BT, Priority for P2: 1
2
4

Enter AT, BT, Priority for P3: 3
4
4

Enter AT, BT, Priority for P4: 4
1
5

Enter AT, BT, Priority for P5: 5
6
2

Enter AT, BT, Priority for P6: 6
5
6

Enter AT, BT, Priority for P7: 10
1
1

1. Non-Preemptive Priority Scheduling
2. Preemptive Priority Scheduling
Enter your choice: 2

Gantt Chart:
| P1 | P5 | P7 | P5 | P1 | P2 | P3 | P4 | P6 |
```

```
PID     AT      BT      PR      CT      TAT     WT      RT
P1      0       8       3       15      15      7       0
P2      1       2       4       17      16      14      14
P3      3       4       4       21      18      14      14
P4      4       1       5       22      18      17      17
P5      5       6       2       12      7       1       0
P6      6       5       6       27      21      16      16
P7      10      1       1       11      1       0       0

ATAT = 13.71
AWT  = 9.86
ART  = 8.71
```

Non Preemptive:

```
PS C:\Users\SHALINI K A\Desktop\OS LAB> gcc PriorityScheduling.c -o PriorityScheduling
PS C:\Users\SHALINI K A\Desktop\OS LAB> ./PriorityScheduling
Enter number of processes: 7

Enter AT, BT, Priority for P1: 0
8
3

Enter AT, BT, Priority for P2: 1
2
4

Enter AT, BT, Priority for P3: 3
4
4

Enter AT, BT, Priority for P4: 4
1
5

Enter AT, BT, Priority for P5: 5
6
2

Enter AT, BT, Priority for P6: 6
5
6

Enter AT, BT, Priority for P7: 10
1
1

1. Non-Preemptive Priority Scheduling
2. Preemptive Priority Scheduling
Enter your choice: 1

Gantt Chart:
| P1 | P5 | P7 | P2 | P3 | P4 | P6 |
```

```
PID     AT      BT      PR      CT      TAT     WT      RT
P1      0       8       3       8       8       0       0
P2      1       2       4       17      16      14      14
P3      3       4       4       21      18      14      14
P4      4       1       5       22      18      17      17
P5      5       6       2       14      9       3       3
P6      6       5       6       27      21      16      16
P7      10      1       1       15      5       4       4

ATAT = 13.57
AWT  = 9.71
ART  = 9.71
```

Priority Scheduling (Preemptive)

```
#include<stdio.h>
int main(){
    int n,i, time=0, remain, highest;
    int at[20], bt[20], rt[20], pr[20];
    int ct[20], tat[20], wt[20], resp[20];
    float avg_tat=0, avg_wt=0, avg_rt=0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Arrival time, Burst time & Priority: \n");
    for(i=0; i<n; i++){
        printf("P%d AT:", i+1);
        scanf("%d", &at[i]);
        printf("P%d BT:", i+1);
        scanf("%d", &bt[i]);
        printf("P%d Priority: ", i+1);
        scanf("%d", &pr[i]);
        rt[i]=bt[i];
        resp[i]=-1;
    }
    remain=n;
    while(remain>0){
        highest=-1;
        for(i=0; i<n; i++){
            if(at[i]<=time && rt[i]>0){
                if(highest==-1 || pr[i]<pr[highest])
                    highest=i;
            }
        }
        if(highest==-1){
            time++;
            continue;
        }
        if(resp[highest]==-1)
            resp[highest]=time-at[highest];
        rt[highest]--;
        time++;
        if(rt[highest]==0){
            ct[highest]=time;
            tat[highest]=ct[highest]-at[highest];
            wt[highest]=tat[highest]-bt[highest];
            remain--;
        }
    }
    printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
    for(i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i], resp[i]);
        avg_tat += tat[i];
        avg_wt += wt[i];
        avg_rt += resp[i];
    }
    printf("\n Average TAT =%.2f", avg_tat/n);
    printf("\nAverage WT =%.2f", avg_wt/n);
    printf("\nAverage RT =%.2f", avg_rt/n);
    return 0;
}
```

OUTPUT

```
Enter the number of Process: 7
Enter the Arrival time & Burst time & Priority:
P1 AT: 0   BT: 8   Priority: 3
P2 AT: 1   BT: 2   Priority: 4
P3 AT: 3   BT: 4   Priority: 4
P4 AT: 4   BT: 1   Priority: 5
P5 AT: 5   BT: 6   Priority: 2
P6 AT: 6   BT: 5   Priority: 6
P7 AT: 10  BT: 1   Priority: 1
```

Process	AT	BT	Priority	CT	TAT	WT	RT
P1	0	8	3	15	15	7	0
P2	1	2	4	17	16	14	14
P3	3	4	4	21	18	14	14
P4	4	1	5	22	18	17	17
P5	5	6	2	12	7	1	0
P6	6	5	6	27	21	16	16
P7	10	1	1	11	1	0	0

Ready queue

P1	P2	P1	P3	P4	P5	P6	P7

Gantt chart

P1	P1	P1	P1	P5	P5	P7	P5	P1	P2	P3	P4	P6

0 1 3 4 5 6 10 11 12 15 17 21 22 27

```
Average TAT = 13.7
Average WT = 9.85
Average RT = 8.71
```

Non-Preemptive

```
Enter the number of Process: 7
Enter the arrival time & Burst time & Priority:
P1 AT: 0   BT: 8   Priority: 3
P2 AT: 1   BT: 2   Priority: 4
P3 AT: 3   BT: 4   Priority: 4
P4 AT: 4   BT: 1   Priority: 5
P5 AT: 5   BT: 6   Priority: 2
P6 AT: 6   BT: 5   Priority: 6
P7 AT: 10  BT: 1   Priority: 1
```

Process	AT	BT	Priority	CT	TAT	WT	RT
P1	0	8	3	8	8	0	0
P2	1	2	4	17	16	14	14
P3	3	4	4	21	18	14	14
P4	4	1	5	22	18	17	17
P5	5	6	2	14	9	3	3
P6	6	5	6	27	21	16	16
P7	10	1	1	15	5	4	4

Ready queue

P1 P2 P3 P4 P5 P6 P7

Gantt chart

P1	P5	P7	P2	P3	P4	P6

0 8 14 15 17 21 22 27

ATAT $= \frac{95}{7} = 13.57$ AWT $= \frac{68}{7} = 9.7$ ART $= \frac{68}{7} = 9.7$